



VISUAL PROGRAMMING

Event-Driven Programming

MUSBAH J. MOUSA
IYAS A. I. ESHAIKHKHALIL

OUTLINES

- **Procedural vs. Event-Driven Programming.**
- **Events:**
 - Sources
 - Classes
 - Information.
- **Handler classes.**
- **Register handler objects.**
- **Defining Handler Classes using:**
 - Inner classes
 - Anonymous classes
 - Lambda expressions.
- **MouseEvents**
- **KeyEvents**
- **Observable Objects Listeners.**

PROCEDURAL VS. EVENT-DRIVEN PROGRAMMING

- Procedural programming is executed in a procedural order.
- In event-driven programming, code is executed upon activation of events.

PROCEDURAL PROGRAMMING

Code



```
String name = "Java";  
String welcomeMessage = String.format("Hello %s!", name);  
System.out.println(welcomeMessage);
```

Output

PROCEDURAL PROGRAMMING

Code



```
String name = "Java"; Executed  
String welcomeMessage = String.format("Hello %s!", name);  
System.out.println(welcomeMessage);
```

Output

PROCEDURAL PROGRAMMING

Code



```
String name = "Java"; Executed  
String welcomeMessage = String.format("Hello %s!", name); Executed  
System.out.println(welcomeMessage);
```

Output

PROCEDURAL PROGRAMMING

Code

```
String name = "Java"; Executed  
String welcomeMessage = String.format("Hello %s!", name); Executed  
System.out.println(welcomeMessage); Executed
```

Output

Hello Java!

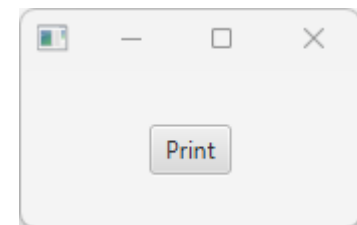
EVENT-DRIVEN PROGRAMMING

Code



```
String name = "Java";  
String welcomeMessage = String.format("Hello %s!", name);  
Button btPrint = new Button("Print");  
btPrint.setOnAction(e -> {System.out.println(welcomeMessage);})
```

Output

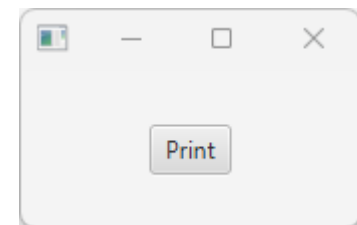


EVENT-DRIVEN PROGRAMMING

Code

 ~~String name = "Java";~~ Executed
String welcomeMessage = String.format("Hello %s!", name);
Button btPrint = new Button("Print");
btPrint.setOnAction(e -> {System.out.println(welcomeMessage);})

Output

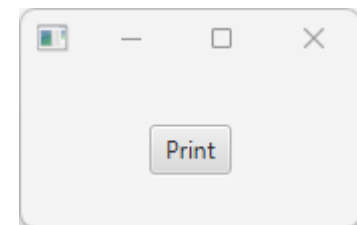


EVENT-DRIVEN PROGRAMMING

Code

```
String name = "Java"; Executed  
String welcomeMessage = String.format("Hello %s!", name); Executed  
➡ Button btPrint = new Button("Print");  
btPrint.setOnAction(e -> {System.out.println(welcomeMessage);})
```

Output

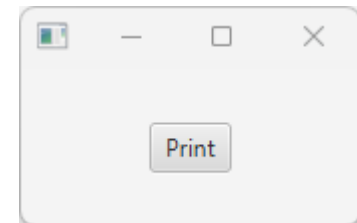


EVENT-DRIVEN PROGRAMMING

Code

~~String name = "Java"; Executed~~
~~String welcomeMessage = String.format("Hello %s!", name); Executed~~
~~Button btPrint = new Button("Print"); Executed~~
→ btPrint.setOnAction(e -> {System.out.println(welcomeMessage);})

Output



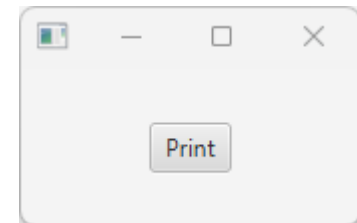
EVENT-DRIVEN PROGRAMMING

Code

```
String name = "Java"; Executed  
String welcomeMessage = String.format("Hello %s!", name); Executed  
Button btPrint = new Button("Print"); Executed  
btPrint.setOnAction(e -> {System.out.println(welcomeMessage);}) Executed
```

Nothing
printed!

Output



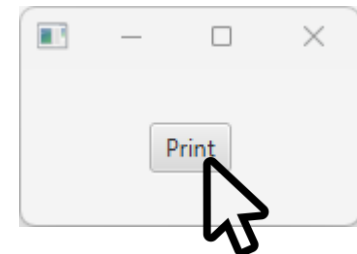
EVENT-DRIVEN PROGRAMMING

Code

```
String name = "Java"; Executed  
String welcomeMessage = String.format("Hello %s!", name); Executed  
Button btPrint = new Button("Print"); Executed  
btPrint.setOnAction(e -> {System.out.println(welcomeMessage);}) Executed
```

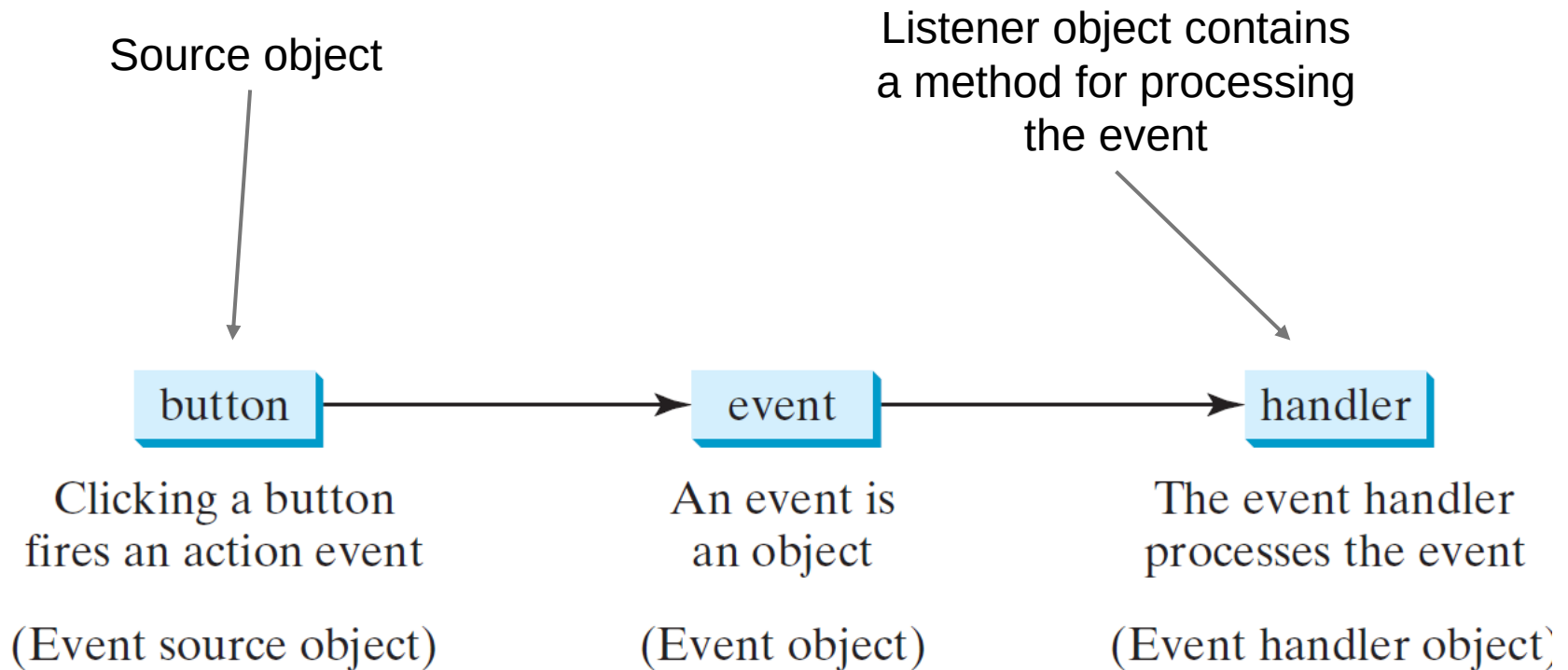
Output

Hello Java!



Click!

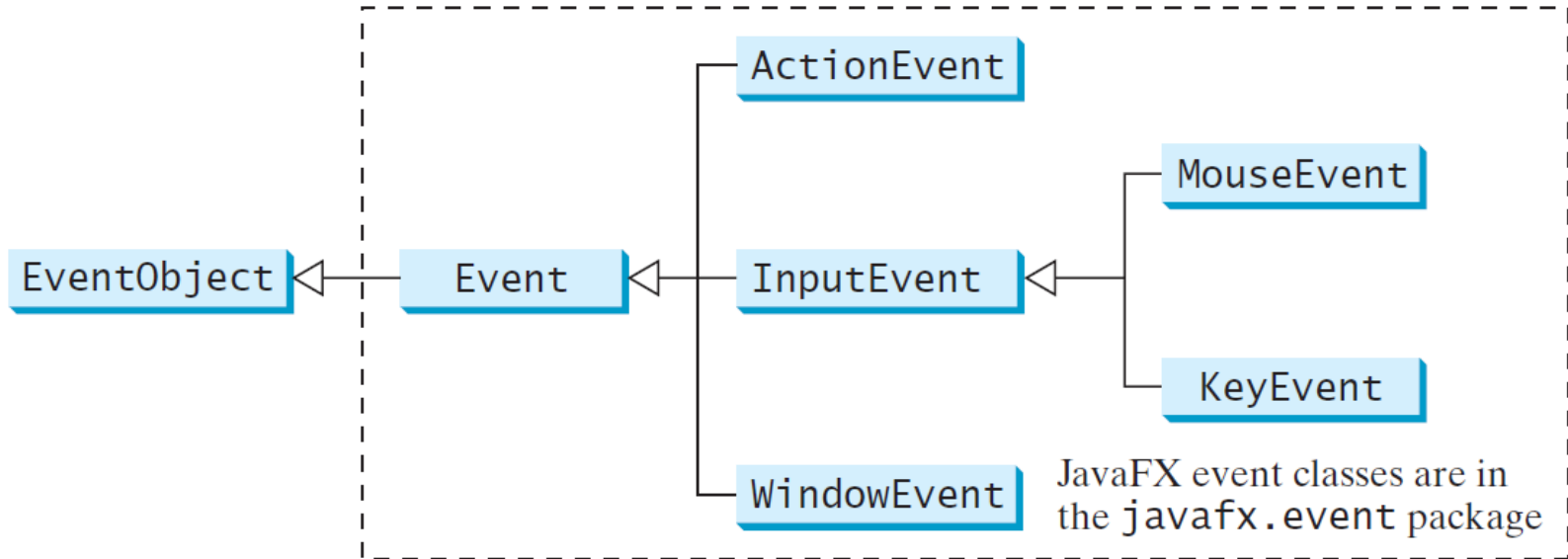
HANDLING GUI EVENTS



EVENTS

- An event can be defined as a type of signal to the program that something has happened.
- The event is generated by external user actions such as mouse movements, mouse clicks, or keystrokes.

EVENT CLASSES



EVENT INFORMATION

- **An event object contains whatever properties are pertinent to the event.**
- **You can identify the source object of the event using the `getSource()` instance method in the `EventObject` class.**
- **The subclasses of `EventObject` deal with special types of events, such as button actions, window events, mouse movements, and keystrokes.**

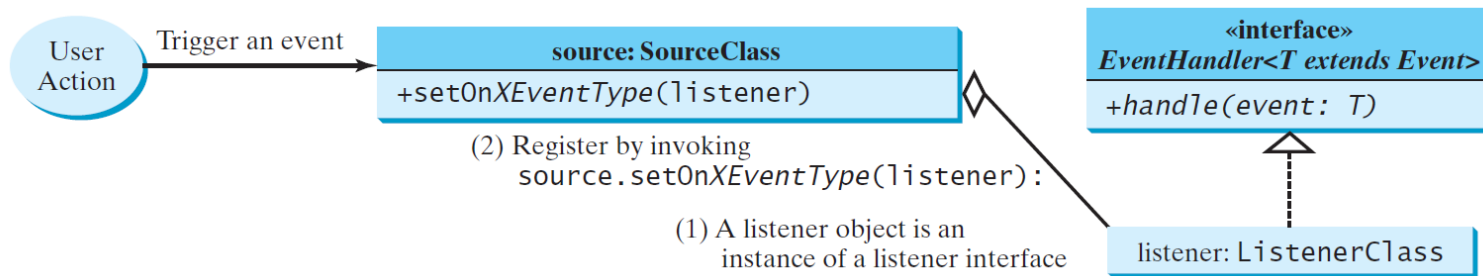
SELECTED USER ACTIONS AND HANDLERS

<i>User Action</i>	<i>Source Object</i>	<i>Event Type Fired</i>	<i>Event Registration Method</i>
Click a button	Button	ActionEvent	setOnAction(EventHandler<ActionEvent>)
Press Enter in a text field	TextField	ActionEvent	setOnAction(EventHandler<ActionEvent>)
Check or uncheck	RadioButton	ActionEvent	setOnAction(EventHandler<ActionEvent>)
Check or uncheck	CheckBox	ActionEvent	setOnAction(EventHandler<ActionEvent>)
Select a new item	ComboBox	ActionEvent	setOnAction(EventHandler<ActionEvent>)
Mouse pressed	Node, Scene	MouseEvent	setOnMousePressed(EventHandler<MouseEvent>)
Mouse released			setOnMouseReleased(EventHandler<MouseEvent>)
Mouse clicked			setOnMouseClicked(EventHandler<MouseEvent>)
Mouse entered			setOnMouseEntered(EventHandler<MouseEvent>)
Mouse exited			setOnMouseExited(EventHandler<MouseEvent>)
Mouse moved			setOnMouseMoved(EventHandler<MouseEvent>)
Mouse dragged			setOnMouseDragged(EventHandler<MouseEvent>)
Key pressed		KeyEvent	setOnKeyPressed(EventHandler<KeyEvent>)
Key released			setOnKeyReleased(EventHandler<KeyEvent>)
Key typed			setOnKeyTyped(EventHandler<KeyEvent>)

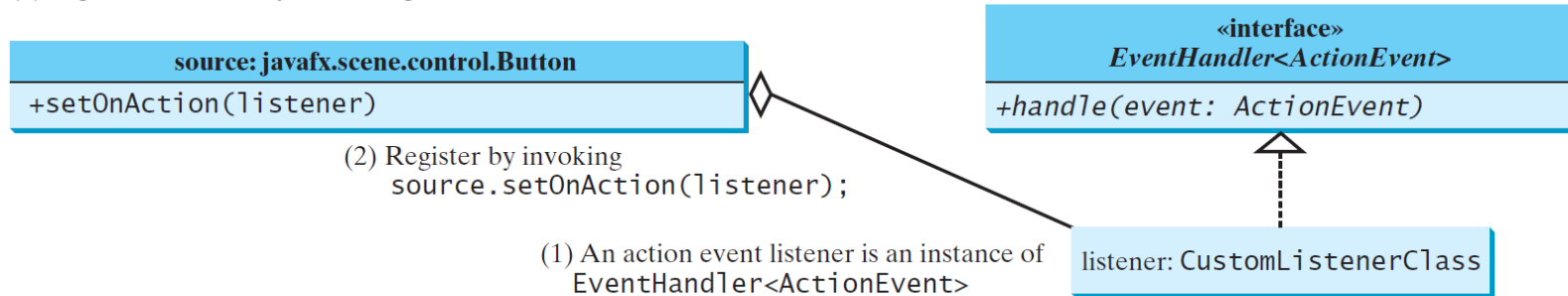
THE DELEGATION MODEL

- Delegation is a model where an object forwards a specific task or responsibility to another object without itself performing the task.
- In other words, delegation involves passing on the responsibility of a certain task to another object, which is better equipped to handle it.

THE DELEGATION MODEL



(a) A generic source object with a generic event T



(b) A Button source object with an ActionEvent

THE DELEGATION MODEL: EXAMPLE

Example

```
Button btOK = new Button("OK");  
OKHandlerClass handler = new OKHandlerClass();  
btOK.setAction(handler);
```

INNER CLASS LISTENERS

- A listener class is designed specifically to create a listener object for a GUI component (e.g., a button).
- It will not be shared by other applications.
- So, it is appropriate to define the listener class inside the frame class as an inner class.

INNER CLASSES

- **Inner class:** A class is a member of another class.
- **Advantages:** In some applications, you can use an inner class to make programs simple.
- An inner class can reference the data and methods defined in the outer class in which it nests, so you do not need to pass the reference of the outer class to the constructor of the inner class.

INNER CLASSES

- **An inner class can be declared public, protected, or private subject to the same visibility rules applied to a member of the class.**
- **An inner class can be declared static. A static inner class can be accessed using the outer class name. A static inner class cannot access non-static members of the outer class**

INNER CLASSES

```
public class Test {  
    ...  
}  
  
public class A {  
    ...  
}
```

(a)

```
public class Test {  
    ...  
  
    // Inner class  
    public class A {  
        ...  
    }  
}
```

(b)

```
// OuterClass.java: inner class demo  
public class OuterClass {  
    private int data;  
  
    /** A method in the outer class */  
    public void m() {  
        // Do something  
    }  
  
    // An inner class  
    class InnerClass {  
        /** A method in the inner class */  
        public void mi() {  
            // Directly reference data and method  
            // defined in its outer class  
            data++;  
            m();  
        }  
    }  
}
```

(c)

ANONYMOUS INNER CLASSES

- Inner class listeners can be shortened using anonymous inner classes.
- An anonymous inner class is an inner class without a name.
- It combines declaring an inner class and creating an instance of the class in one step.
- It's a way to create a one-time use class that can be used to define a behavior that is only needed in a specific context or for a limited time.

ANONYMOUS INNER CLASSES

- An anonymous inner class is declared as follows:

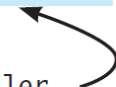
```
new SuperClassName/InterfaceName() {  
    // Implement or override methods in superclass or interface  
    // Other methods if necessary  
}
```

ANONYMOUS INNER CLASSES

- An anonymous inner class must always extend a superclass or implement an interface, but it cannot have an explicit extends or implements clause.
- An anonymous inner class must implement all the abstract methods in the superclass or in the interface.

ANONYMOUS INNER CLASSES

```
public void start(Stage primaryStage) {  
    // Omitted  
  
    btEnlarge.setOnAction(  
        new EnlargeHandler());  
}  
  
class EnlargeHandler  
    implements EventHandler<ActionEvent> {  
    public void handle(ActionEvent e) {  
        circlePane.enlarge();  
    }  
}
```



(a) Inner class EnlargeListener

```
public void start(Stage primaryStage) {  
    // Omitted  
  
    btEnlarge.setOnAction(  
        new class EnlargeHandler  
            implements EventHandler<ActionEvent>() {  
                public void handle(ActionEvent e) {  
                    circlePane.enlarge();  
                }  
            }  
    );  
}
```

(b) Anonymous inner class

SIMPLIFYING EVENT HANDLING USING LAMBDA EXPRESSIONS

- Lambda expression is a new feature in Java 8.
- Lambda expressions can be viewed as an anonymous method with a concise syntax.
- For example, the following code in (a) can be greatly simplified using a lambda expression in (b) in three lines.

```
btEnlarge.setAction(  
    new EventHandler<ActionEvent>() {  
        @Override  
        public void handle(ActionEvent e) {  
            // Code for processing event e  
        }  
    }  
);
```

(a) Anonymous inner class event handler

```
btEnlarge.setAction(e -> {  
    // Code for processing event e  
});
```

(b) Lambda expression event handler

BASIC SYNTAX FOR A LAMBDA EXPRESSION

- The basic syntax for a lambda expression is either

`(type1 param1, type2 param2, ...) -> expression`

or

`(type1 param1, type2 param2, ...) -> { statements; }`

- The data type for a parameter may be explicitly declared or implicitly inferred by the compiler.
- The parentheses can be omitted if there is only one parameter without an explicit data type.

SINGLE ABSTRACT METHOD INTERFACE (SAM)

- The statements in the lambda expression is all for that method.
- If it contains multiple methods, the compiler will not be able to compile the lambda expression.
- So, for the compiler to understand lambda expressions, the interface **must** contain exactly one abstract method.
- Such an interface is known as a functional interface, or a Single Abstract Method (SAM) interface.

THE MouseEvent CLASS

`javafx.scene.input.MouseEvent`

```
+getButton(): MouseButton  
+getClickCount(): int  
+getX(): double  
+getY(): double  
+getSceneX(): double  
+getSceneY(): double  
+getScreenX(): double  
+getScreenY(): double  
+isAltDown(): boolean  
+isControlDown(): boolean  
+isMetaDown(): boolean  
+isShiftDown(): boolean
```

Indicates which mouse button has been clicked.

Returns the number of mouse clicks associated with this event.

Returns the *x*-coordinate of the mouse point in the event source node.

Returns the *y*-coordinate of the mouse point in the event source node.

Returns the *x*-coordinate of the mouse point in the scene.

Returns the *y*-coordinate of the mouse point in the scene.

Returns the *x*-coordinate of the mouse point in the screen.

Returns the *y*-coordinate of the mouse point in the screen.

Returns true if the **Alt** key is pressed on this event.

Returns true if the **Control** key is pressed on this event.

Returns true if the mouse **Meta** button is pressed on this event.

Returns true if the **Shift** key is pressed on this event.

THE MouseEvent CLASS

- **Four constants are defined in MouseButton:**
 - PRIMARY => left button
 - SECONDARY => right button
 - MIDDLE => middle button
 - NONE => none button
- **You can use the `getButton()` method to detect which button is pressed.**
- **For example, `getButton() == MouseButton.SECONDARY` tests if the right button was pressed.**
- **You can also use the following methods:**
 - `isPrimaryButtonDown()`
 - `isSecondaryButtonDown()`
 - `isMiddleButtonDown()`

THE MouseEvent CLASS

`javafx.scene.input.KeyEvent`

```
+getCharacter(): String  
+getCode(): KeyCode  
+getText(): String  
+isAltDown(): boolean  
+isControlDown(): boolean  
+isMetaDown(): boolean  
+isShiftDown(): boolean
```

Returns the character associated with the key in this event.

Returns the key code associated with the key in this event.

Returns a string describing the key code.

Returns true if the `Alt` key is pressed on this event.

Returns true if the `Control` key is pressed on this event.

Returns true if the mouse `Meta` button is pressed on this event.

Returns true if the `Shift` key is pressed on this event.

KeyEvent

- The **key pressed** handler is invoked when a **key is pressed**.
- The **key released** handler is invoked when a **key is released**.
- The **key typed** handler is invoked when a **Unicode character is entered**.
- If a **key does not have a Unicode** (e.g., function keys, modifier keys, action keys, arrow keys, and control keys), the **key typed handler will not be invoked**.

KeyEvent

- Every key event has an associated code that is returned by the `getCode()` method in `KeyEvent`.
- **For the key-pressed and key-released events:**
 - `getCode()` returns the value as defined in the table.
 - `getText()` returns a string that describes the key code.
 - `getCharacter()` returns an empty string.
- **For the key-typed event:**
 - `getCode()` returns `UNDEFINED`.
 - `getCharacter()` returns the Unicode character or a sequence of characters associated with the key-typed event.

THE `keyCode` CONSTANTS

<i>Constant</i>	<i>Description</i>	<i>Constant</i>	<i>Description</i>
<code>HOME</code>	The Home key	<code>CONTROL</code>	The Control key
<code>END</code>	The End key	<code>SHIFT</code>	The Shift key
<code>PAGE_UP</code>	The Page Up key	<code>BACK_SPACE</code>	The Backspace key
<code>PAGE_DOWN</code>	The Page Down key	<code>CAPS</code>	The Caps Lock key
<code>UP</code>	The up-arrow key	<code>NUM_LOCK</code>	The Num Lock key
<code>DOWN</code>	The down-arrow key	<code>ENTER</code>	The Enter key
<code>LEFT</code>	The left-arrow key	<code>UNDEFINED</code>	The <code>keyCode</code> unknown
<code>RIGHT</code>	The right-arrow key	<code>F1</code> to <code>F12</code>	The function keys from F1 to F12
<code>ESCAPE</code>	The Esc key	<code>0</code> to <code>9</code>	The number keys from 0 to 9
<code>TAB</code>	The Tab key	<code>A</code> to <code>Z</code>	The letter keys from A to Z

FOCUS

- Only a focused node can receive KeyEvent .
- Invoking `requestFocus()` on text enables text to receive key input.
- This method must be invoked after the stage is displayed.

LISTENERS FOR OBSERVABLE OBJECTS

- You can add a listener to process a value change in an observable object.
- An instance of **Observable** is known as an **observable object**, which contains the `addListener(InvalidationListener listener)` method for adding a listener.
- Once the value is changed in the property, a listener is notified.
- The listener class should implement the **InvalidationListener** interface, which uses the `invalidated(Observable o)` method to handle the property value change.
- Every binding property is an instance of **Observable**.

ANY QUESTIONS?